

Лекция 12. Итоговый инфраструктурный проект Linux-инфраструктуры

Цель лекции

Понять, как отдельные технологии PO2.3
объединяются в работающую Linux-инфраструктуру
Проектировать роли серверов, имена, адресацию,
сетевые зоны и зависимости сервисов
Выбирать безопасный порядок развертывания с Git,
backup и monitoring перед изменениями
Проверять инфраструктуру через сценарии доступа,
отказа, восстановления и перезагрузки
Готовить документацию, по которой другой
администратор сможет сопровождать стенд

Учебные вопросы

1. Многосерверная Linux-инфраструктура, роли, адресация и naming
2. Зависимости DNS, VPN, web, mail, monitoring и backup
3. Production-, management- и backup-трафик
4. Безопасный порядок развертывания, Git, мониторинг и backup перед изменениями
5. Диагностика ресурсов, ОС, сети, службы и приложения
6. Эксплуатационные, восстановительные и исполнительные документы

Учебный сценарий лекции

Студент готовит итоговый стенд для условной организации с доменом `company.test`

Стенд должен включать удаленный доступ, web-сервис, мониторинг, backup и документацию

Почта, Docker и отдельная backup network могут входить в полный или расширенный вариант

Проект оценивается не по факту установки служб, а по работе сценариев и восстановлению

Все имена, адреса, правила firewall и проверки должны совпадать с документацией

Обязательная и расширенная части уменьшают перегруз проекта

Обязательное ядро: DNS, VPN, HAProxy, два web backend, Zabbix, backup, Git и Bash health check

Расширенная часть: Mailcow, отдельная backup network, Docker application, Keepalived и автоматизация

В индивидуальном проекте расширенная часть может оцениваться дополнительными баллами

Формат работы заранее фиксируется: индивидуально, в паре или в команде

Такой подход сохраняет профессиональный масштаб и делает оценивание объективным

1. Итоговый проект проверяет инфраструктуру как систему

Инфраструктура — это связанные серверы, сети, данные, правила доступа и процедуры сопровождения

Отдельно установленный сервис не считается завершенным без проверки зависимостей

Сервис должен запускаться после перезагрузки и восстанавливаться после типовой ошибки

Адресный план, DNS, firewall и мониторинг должны описывать один и тот же стенд

Итоговая защита показывает работу системы, а не набор скриншотов

Полный список узлов делает проект проверяемым

client01 — пользовательский или VPN-клиент для проверки доступа к сервисам

admin01 — административная рабочая станция для SSH, Git и management-доступа

dns01 — внутренний DNS-сервер зоны company.test

vpn01, lb01, web01 и web02 обеспечивают удаленный доступ и web-сервис

mail01, zabbix01, backup01 и git01 используются по выбранному варианту проекта

Naming standard устраняет путаницу между серверами

FQDN (Fully Qualified Domain Name) — полное доменное имя узла, например `web01.company.test`

Правило именования: роль плюс номер, затем домен `company.test`

DNS-запись, `hostname`, имя в Zabbix и имя в документации должны совпадать

Для почты можно использовать `hostname mail01` и сервисное имя `mail.company.test`

Нельзя смешивать `mail01.company.test` и `mail.company.test` без явного объяснения

Адресный план показывает не только IP-адрес

Production network: 192.168.30.0/24, пользовательские сервисы и backend-серверы

Management network: 192.168.40.0/24, admin01, dns01, zabbix01 и SSH-доступ

Backup network: 192.168.50.0/24, backup01 и источники резервного копирования

VPN subnet: 10.8.0.0/24, адреса клиентов OpenVPN

В таблице для каждого узла указываются role, IP, gateway, DNS и доступные порты

Отдельная сеть может быть физической или логической

В VirtualBox удобно использовать несколько Internal Network: prod-net, mgmt-net и backup-net

В реальной сети разделение обычно выполняется через VLAN и маршрутизацию

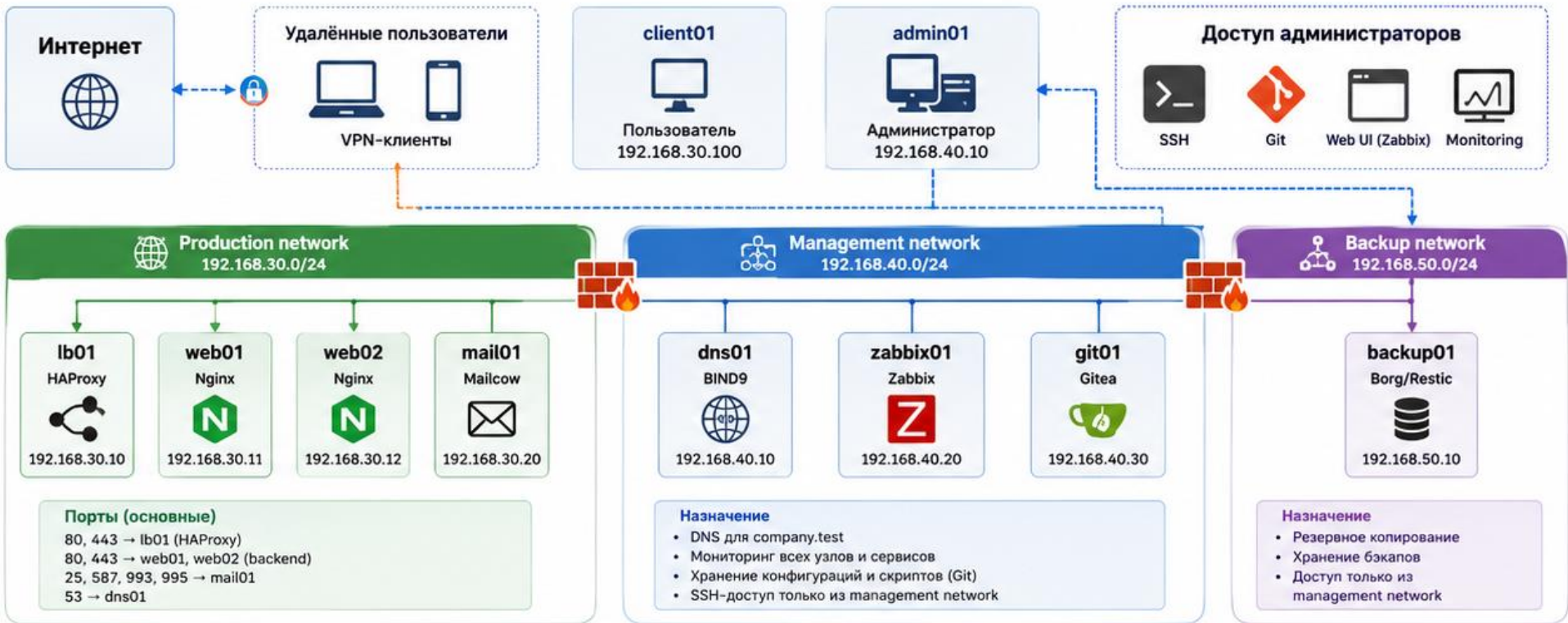
Если NIC мало, допускаются разные подсети и строгие firewall rules на одном маршрутизаторе

Разделение считается выполненным только при наличии правил Source и Destination

Студент должен объяснить, почему выбран именно такой способ разделения

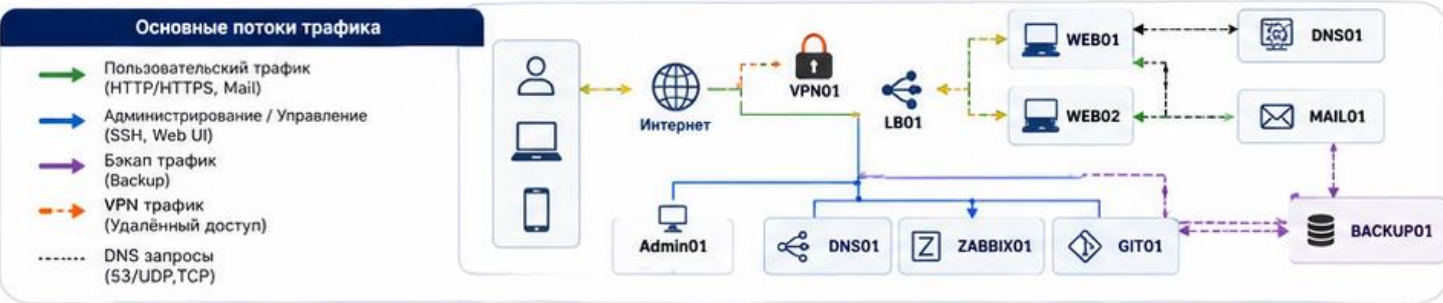
Топология: итоговая архитектура Linux-инфраструктуры

company.test – связанная, безопасная и отказоустойчивая Linux-инфраструктура



Безопасность и трафик

- ✓ Firewall (iptables/nftables): разрешаем только нужные порты между сетями
- ✓ Все сервисы доступны только по необходимым протоколам
- ✓ Администрирование и бэкап – только из management network
- ✓ Регулярный backup и проверка восстановления



Единые принципы: именование, адресация, разделение сетей, минимальные права, мониторинг, резервное копирование, документация

Naming standard

<роль><номер>.company.test
Примеры: web01.company.test, dns01.company.test

Адресный план

Production network	192.168.30.0/24
Management network	192.168.40.0/24
Backup network	192.168.50.0/24
VPN subnet	10.8.0.0/24

Ключевые сервисы

- DNS (dns01)**
Внутренний DNS company.test
- VPN (vpn01)**
OpenVPN для удалённого доступа
- Load Balancer (lb01)**
HAProxy, балансировка и TLS
- Web backend (web01, web02)**
Веб-приложение (Nginx/PHP/Node.js)
- Mail (mail01)**
Почта и антиспам (Mailcow)
- Monitoring (zabbix01)**
Zabbix: мониторинг и алерты
- Backup (backup01)**
Резервное копирование (Borg/Restic)
- Git (git01)**
Git сервер для конфигураций и скриптов

Отказоустойчивость и восстановление

- ✓ LB01 может быть дублирован (Keepalived) для VIP и отказоустойчивости
- ✓ WEB01/WEB02 – горизонтальное масштабирование
- ✓ Ежедневное тестирование восстановления из бэкапа
- ✓ Документация со схемами, адресами, правилами и процедурами

2. DNS должен быть отдельной и понятной ролью

DNS (Domain Name System) — система преобразования имен серверов в IP-адреса

В итоговом стенде DNS разворачивается на dns01 или явно предоставляется преподавателем

Если DNS размещен на другом сервере, это нужно указать как совмещение ролей и SPOF

SPOF (Single Point of Failure) — единая точка отказа, от которой зависит несколько сервисов

Без определенного источника DNS невозможно объективно принять проект

DNS-проверка связывает имена с адресным планом

Команда `resolvectl query` проверяет системное разрешение имени: `$ resolvectl query web01.company.test`

Ожидаемый результат: `web01.company.test` разрешается в адрес из таблицы проекта

Команда `dig` показывает конкретную DNS-запись: `$ dig +short A lb01.company.test`

Для почты MX указывает на имя `mail.company.test`, а не на IP-адрес

Если DNS не работает, сначала проверяют `dns01`, маршрут до него и `firewall`

Матрица зависимостей показывает последствия отказа

OpenVPN зависит от DNS, времени, CA, firewall и routing

HAProxy зависит от backend-серверов, DNS или IP-адресов и открытых портов

Mailcow зависит от Docker, DNS, TLS, disk space, database и времени

Zabbix зависит от базы данных, агентов, сети и актуальных items

Backup зависит от SSH, repository, свободного места, ключей и расписания

Мониторинг общей зависимости должен быть независимым

Если Zabbix проверяет все узлы только по DNS-именам, отказ DNS скрывает реальную картину
dns01 нужно контролировать по IP-адресу и отдельным DNS-запросом

Команда zabbix_get проверяет агент напрямую: \$
zabbix_get -s 192.168.40.53 -k agent.ping

Ожидаемый ответ 1 означает, что агент доступен и item может получать данные

Для критичных сервисов полезен внешний check с client01 или admin01

Зависимости объясняют, почему симптом может быть обманчивым

Пользователь видит web недоступным, но причина может быть в VPN, DNS, HAProxy или backend

Почта может не доставляться из-за TLS, PTR, заполненного диска или контейнера Mailcow

Васкир может считаться успешным, хотя исходный каталог не был смонтирован

Отсутствие Zabbix problems не доказывает исправность, если items перестали обновляться

Диагностика начинается с цепочки зависимостей, а не с случайной перезагрузки



Схема: матрица зависимостей итогового стенда

company.test — взаимосвязи сервисов, данных и сетей

Как читать матрицу

- Зелёная галочка**
Сервис в строке использует сервис в столбце
- Оранжевый круг**
Сервис в строке зависит от данных/ресурсов в столбце
- Фиолетовая стрелка**
Необязательная или условная зависимость
- Серая ячейка**
Нет прямой зависимости

Ключевые принципы

- Минимизация точек отказа
- Чёткий порядок запуска
- Мониторинг и алерты
- Резервное копирование
- Документирование и тесты

Зависит от →

Использует ↓

	DNS dns01	VPN vpn01	HAProxy lb01	Web web01/02	Mail mail01	Zabbix zabbix01	Backup backup01	Git git01	Storage (данные)	ОС и сеть (базовые)
DNS dns01	—	—	—	—	—	—	—	—	—	
VPN vpn01		—	—	—	—	—	—	—	—	
HAProxy lb01			—	—	—	—	—	—	—	
Web web01/02				—	—	—	—	—		
Mail mail01			—	—	—	—	—	—		
Zabbix zabbix01			—	—	—	—	—	—		
Backup backup01			—	—		—	—	—		
Git git01			—	—	—	—	—			
Storage (данные)	—	—	—	—	—	—	—		—	
ОС и сеть (базовые)										—

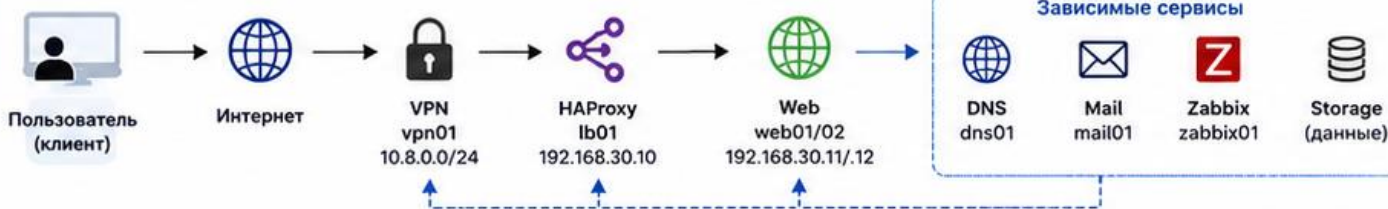
Примеры зависимостей

- Web → HAProxy**
Веб-серверы получают трафик только через балансировщик lb01
- Mail → DNS**
Почтовый сервер использует DNS для MX-записей и поиска доменов
- Backup → Storage**
Система резервного копирования читает и записывает данные в хранилище
- Администрирование → VPN**
Доступ администраторов к серверам только через VPN-шлюз

Порядок запуска (рекомендуемый)

- ОС и сеть (базовые настройки)
- DNS
- VPN
- HAProxy
- Web (web01, web02)
- Mail (если используется)
- Zabbix
- Storage и данные
- Backup
- Git и вспомогательные сервисы

Поток доступа пользователя



Условные обозначения (в матрице)

- Прямая зависимость (нужен для работы)
- Использует данные/ресурсы
- Необязательная / условная зависимость
- Нет прямой зависимости

Важно

- Изменения в одной службе могут повлиять на другие.
- Проверяйте зависимости перед обновлениями.
- Всегда делайте backup перед изменениями.

Матрица помогает понять, какие сервисы критичны, в каком порядке их разворачивать и что проверять при ошибках или отказах.

3. Traffic separation делает безопасность проверяемой

Production traffic — пользовательский доступ к web, mail и разрешенным внутренним ресурсам

Management traffic — SSH, Zabbix, Git, административные web-интерфейсы и управление сервисами

Backup traffic — копирование данных к backup01 и проверка repository

VPN traffic — внешний доступ клиентов к строго ограниченным внутренним направлениям

Разделение трафика подтверждается маршрутами, firewall rules и тестами доступа

Таблица потоков задает разрешенные Source и Destination

VPN Client -> lb01 TCP 443: разрешить

пользовательский доступ к web

VPN Client -> web01 TCP 22: запретить прямое

администрирование backend

admin01 -> servers TCP 22: разрешить SSH только

администраторам

zabbix01 -> agents TCP 10050: разрешить passive checks

при выбранной модели

servers -> backup01 TCP 22: разрешить только по

выбранной push или pull backup-модели

Таблица портов превращает firewall в проверяемую политику

OpenVPN: UDP 1194 от external client к vpn01

HTTPS: TCP 443 от users или VPN к lb01

HAProxy -> Nginx: TCP 80 от lb01 к web01 и web02

Submission и IMAPS: TCP 587 и TCP 993 от clients к mail01

Zabbix active checks: TCP 10051 от agents к zabbix01

VPN проверяется разрешенным и запрещенным доступом

Валидный client certificate должен устанавливать tunnel до vpn01

VPN-клиент должен открыть web через lb01:443

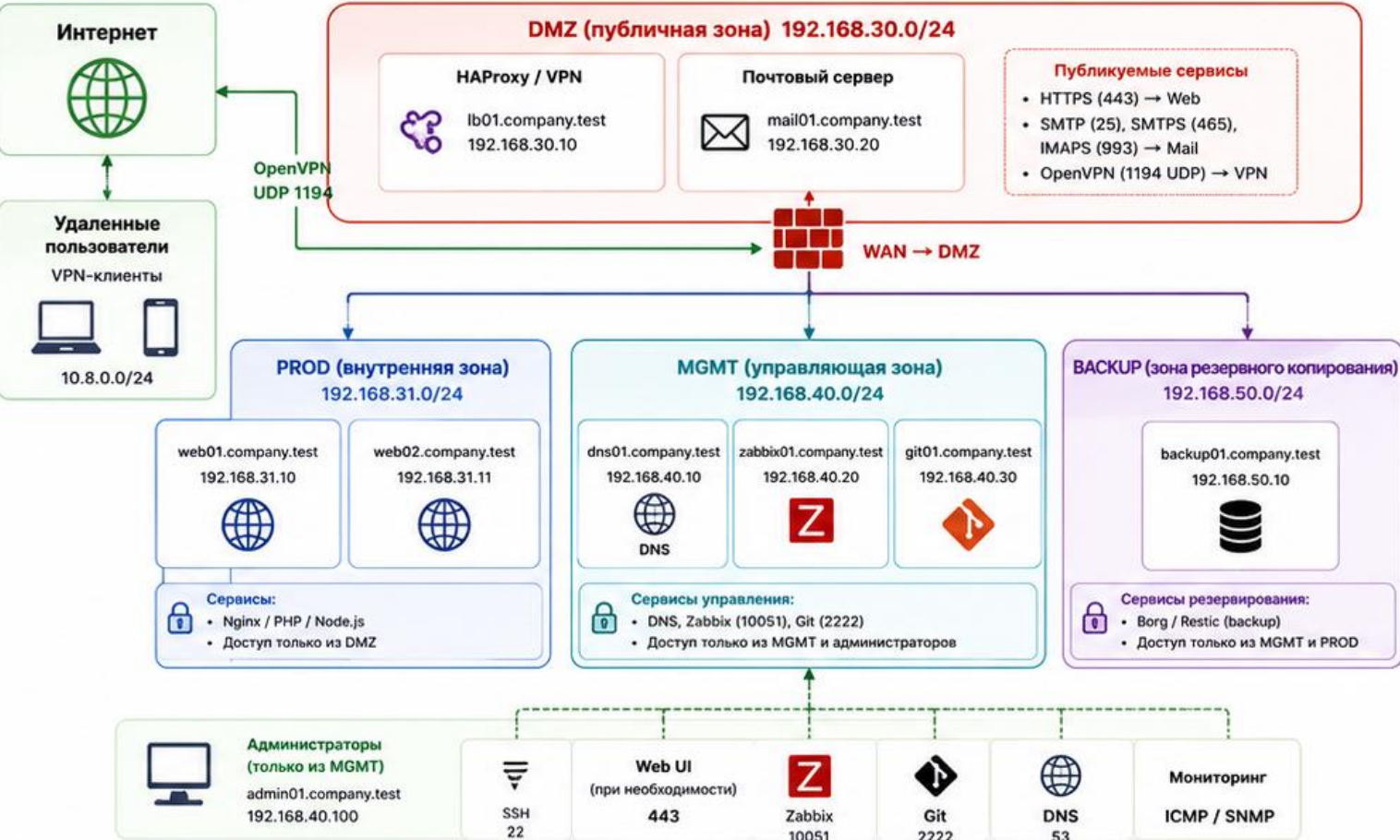
VPN-клиент не должен получать SSH-доступ к web01 и web02

Отозванный certificate должен блокировать подключение до внутренней сети

Без VPN внутреннее имя web-сервиса не должно быть доступно из внешней сети

Схема: сетевые зоны и разрешенные firewall-потоки

company.test — разделяем сети, разрешаем только необходимое



Разрешенные firewall-потоки (минимально необходимые)					
№	Источник (зона)	Назначение (зона/узел)	Сервис / Порт	Протокол	Назначение
1	Интернет	DMZ (lb01)	UDP 1194	UDP	OpenVPN доступ
2	Интернет	DMZ (lb01)	TCP 443	TCP	HTTPS к web
3	Интернет	DMZ (mail01)	TCP 25	TCP	SMTP
4	Интернет	DMZ (mail01)	TCP 465	TCP	SMTPS
5	Интернет	DMZ (mail01)	TCP 993	TCP	IMAPS
6	DMZ (lb01)	PROD (web01, web02)	TCP 80, 443	TCP	HTTP/HTTPS к веб
7	DMZ (mail01)	PROD (web01, web02)	TCP 80, 443	TCP	Web интерфейсы (если нужны)
8	MGMT (admin01)	MGMT (все узлы)	TCP 22	TCP	SSH управление
9	MGMT (admin01)	MGMT (zabbix01)	TCP 10051	TCP	Zabbix агент
10	MGMT (admin01)	MGMT (git01)	TCP 2222	TCP	Git доступ
11	MGMT (dns01)	Все зоны	TCP/UDP 53	TCP/UDP	DNS-запросы
12	PROD (web01,02)	BACKUP (backup01)	TCP 22, 2333*	TCP	Резервное копирование (Borg/Restic по SSH)
13	MGMT (admin01)	BACKUP (backup01)	TCP 22	TCP	Управление backup
14	Zabbix (zabbix01)	Все узлы	ICMP / SNMP	ICMP/UDP	Мониторинг
*	Любой	Любой	Остальные	—	ЗАПРЕЩЕНО (DROP)

Политика по умолчанию: запрещать все, разрешать только явно указанные потоки.
Firewall установлен на lb01 (DMZ) и/или на границе сетей (межсетевой экран).

Сетевые зоны

- DMZ (публичная) – сервисы, доступные из Интернета
- PROD (внутренняя) – рабочие сервисы компании
- MGMT (управляющая) – администрирование и мониторинг
- BACKUP (резервная) – хранение резервных копий
- VPN subnet – адреса подключенных клиентов

Принципы разделения

- Публичные сервисы только в DMZ
- Веб-бэкэнды недоступны из Интернета напрямую
- Управление только из MGMT сети
- Резервные копии изолированы в BACKUP сети
- Минимально необходимые порты и протоколы

Ключевые правила

- Не открываем лишние порты
- Используем TLS для всех чувствительных сервисов
- SSH доступ только из MGMT и по ключам
- Регулярно проверяем логи firewall и сервисов

Быстрая проверка потоков

```
# Проверка из Интернета
nmap -Pn lb01.company.test

# Проверка из PROD к backup
nc -zv backup01.company.test 22

# Проверка DNS
nslookup web01.company.test dns01.company.test
```

Обозначения

- Разрешено
- Публичный доступ
- Внутренний доступ
- Только управление
- Backup-потоки
- Межсетевой экран

Развертывание начинается с проектирования, а не с установки пакет

Сначала утверждаются роли, адреса, имена, сетевые зоны и обязательные сценарии приемки

Затем подготавливаются базовая ОС, SSH, время, DNS и management-доступ

После этого подключаются Git, backup и monitoring, чтобы изменения сразу фиксировались

Сложные сервисы разворачиваются только после появления наблюдаемости и восстановления

Финальный этап включает отказные тесты, reboot test, restore и документацию

Базовая ОС должна иметь устойчивые настройки

Команда `hostnamectl status` показывает hostname и версию ОС: `$ hostnamectl status`

Ожидаемый hostname должен совпадать с naming standard и DNS-записью

Команда `timedatectl status` проверяет синхронизацию времени: `$ timedatectl status`

Для TLS, VPN, Kerberos-подобных механизмов и журналов важно корректное системное время

Сетевые адреса проверяются кратко: `$ ip -br addr`

Git фиксирует изменение только после проверки

Команда `git status` показывает измененные файлы: `$ git status`

Перед `commit` администратор читает изменения: `$ git diff`

После `staging` нужно проверить подготовленное: `$ git diff --staged`

`Commit-message` должен объяснять сервис и смысл изменения

Нельзя коммитить конфигурацию, если она не прошла `syntax check`

Secret Check выполняется до git add

Secret — пароль, private key, token, .env, VPN profile или backup password

Команда git status не показывает секреты внутри файла, поэтому нужен ручной просмотр diff

Команда git diff помогает увидеть случайно добавленный пароль до фиксации

Файлы с секретами заменяются шаблонами с CHANGE_ME или REDACTED

.gitignore не удаляет секрет, который уже попал в историю репозитория

Васкир перед изменением должен быть проверенным

RPO (Recovery Point Objective) — допустимая потеря данных по времени

Команда `restic snapshots` показывает доступные точки восстановления: `$ restic snapshots`

Команда `restic check` проверяет целостность repository: `$ restic check`

Нужно убедиться, что `source` был смонтирован и backup содержит нужные данные

Для критичных сервисов ранее должен быть выполнен `test restore`

itoring после изменения проверяет свежие данные, а не только Prob

Отсутствие Problems не доказывает исправность, если items перестали получать данные

Нужно проверить свежесть метрик CPU, RAM, disk, network и service availability

Unsupported Item означает, что проверка есть, но фактически не работает

Host в Maintenance может скрыть настоящую проблему

После изменения обязательно выполняется пользовательский external test

RAID Failure Test выполняется безопасно

RAID (Redundant Array of Independent Disks) — массив дисков с выбранным уровнем отказоустойчивости

Отказ диска нельзя демонстрировать на основном хранилище критичных сервисов проекта

Лучше использовать отдельную storage VM или отдельный небольшой тестовый массив

Перед тестом нужен snapshot или заранее подготовленный учебный стенд

В основной инфраструктуре достаточно показать состояние массива и документацию rebuild



Диаграмма: безопасный порядок развертывания итогового стенда

company.test — сначала проектирование и наблюдаемость, потом сервисы и проверка

1. Проектирование



- Роли серверов
- Имена и FQDN
- Адресный план
- Сетевые зоны
- Сценарии приемки

2. Базовая ОС и доступ



- Установка ОС
- Hostname и IP
- SSH-доступ
- Время / NTP
- DNS и маршруты

3. Подготовка к изменениям



- Git для конфигураций
- Backup перед изменением
- Monitoring и Zabbix
- Health checks
- Secret check

4. Базовые инфраструктурные сервисы



dns01



vpn01



lb01 / HAProxy



Storage / mount



Firewall rules

5. Сервисы приложений



web01 / web02



Mailcow / Docker



Git service



Backup service



Только после наблюдаемости

6. Проверка после изменений



Syntax check



Проверка портов



Свежесть метрик



External test



Rollback при ошибке

7. Приемка и сопровождение



- Acceptance tests
- Failover test
- Reboot test
- Test restore
- Документация



А. Ключевой принцип

- ✓ Не ставим сложные сервисы первыми
- ✓ Наблюдаемость раньше изменений
- ✓ Backup без restore — не гарантия
- ✓ Документация должна совпадать со стендом



В. Перед Git commit

- ☐ git diff
- ☐ syntax check
- ☐ секреты не попали в файлы
- ☐ есть backup
- ☐ понятный commit message



С. Финальная проверка

- ✓ DNS работает
- ✓ VPN дает только разрешенный доступ
- ✓ Web доступен через lb01
- ✓ Zabbix показывает актуальные данные
- ✓ Стенд переживает перезагрузку

Безопасный цикл изменения



Изменение



Проверка



Мониторинг



Backup/Restore



Документация

5. Диагностика начинается с клиентского пути

Пользовательский путь: client -> DNS ->
route/firewall/VPN -> entry point -> service ->
dependency -> data

Если приложение работает локально, но недоступно
пользователю, проверяется путь от клиента

Если путь работает, но функция нарушена,
проверяются приложение, данные и зависимости

Такой порядок уменьшает хаотичные перезапуски
служб

В отчете фиксируются симптом, гипотеза, проверка,
исправление и подтверждение

Ресурсы и ОС проверяются до сложной диагностики

Команда uptime показывает load average: `$ uptime`

Команда free -h помогает увидеть нехватку RAM и swap: `$ free -h`

Команда df -h показывает заполнение файловых систем: `$ df -h`

Команда df -i выявляет исчерпание inode: `$ df -i`

Команда systemctl --failed показывает службы, которые systemd считает отказавшими

Сеть проверяется по адресу, маршруту, DNS и firewall

Команда `ip -br addr` показывает адреса интерфейсов: `$ ip -br addr`

Команда `ip route` показывает gateway и маршруты: `$ ip route`

Команда `resolvectl query` проверяет DNS глазами системы: `$ resolvectl query lb01.company.test`

Команда `ss -ltnp` показывает слушающие TCP и UDP порты: `$ sudo ss -ltnp`

Команда `nft list ruleset` помогает увидеть активные firewall rules: `$ sudo nft list ruleset`

Служба и приложение проверяются разными командами

Команда `systemctl status nginx` показывает состояние службы: `$ systemctl status nginx`

Команда `journalctl` читает журнал службы: `$ sudo journalctl -u nginx -n 50`

Команда `nginx -t` проверяет синтаксис Nginx до перезапуска: `$ sudo nginx -t`

Команда `haproxy -c` проверяет конфигурацию балансировщика: `$ sudo haproxy -c -f /etc/haproxy/haproxy.cfg`

Команда `docker compose ps` показывает состояние контейнеров в текущем compose-проекте

External Test показывает реальный пользовательский результат

Проверка с сервера не заменяет проверку с client01 или admin01

Команда curl проверяет HTTP-ответ через entry point: \$
curl -I https://web.company.test

Ожидаемый результат: код 200, 301 или другой
заранее объясненный статус

Для Mailcow внутренний тест: user1@company.test
отправляет письмо user2@company.test

Успех подтверждается доставкой, пустой deferred
queue и журналом без критичных ошибок

Отказный тест должен быть безопасным и обратимым

Предусловия показывают, что остальные компоненты готовы к демонстрации

Действие отказа должно затрагивать только один заранее выбранный компонент

Ожидаемый симптом и Zabbix problem должны быть известны заранее

Recovery возвращает сервис в штатное состояние и закрывает проблему

После восстановления повторяется пользовательский сценарий

Отозванный VPN-сертификат — это контроль безопасности

Revoked certificate не является поломкой VPN-сервера

Это ожидаемый результат security policy, если клиент больше не должен подключаться

Пользователь может сообщить, что web01 недоступен, но причина находится до входа в tunnel

Диагностика проверяет VPN connection, certificate status, route и firewall

Успешный результат: валидный клиент подключается, отозванный клиент отклоняется

Web/HAProxy failure проверяется без остановки всей системы

Перед тестом нужно убедиться, что web02 работает и HAProxy видит оба backend

Команда останавливает только web01: `$ sudo systemctl stop nginx`

Ожидаемый результат: HAProxy помечает web01 как DOWN и продолжает отдавать web02

После теста служба возвращается: `$ sudo systemctl start nginx`

Recovery подтверждается curl-тестом, HAProxy status и закрытием Zabbix problem

Mailcow-сценарий должен быть конкретным

Отправитель: user1@company.test; получатель:
user2@company.test

Способ отправки: Webmail или Submission 587 с
аутентификацией

Ожидаемый результат: письмо появляется у user2 не
позднее 60 секунд

Очередь не должна содержать Deferred Message после
внутренней доставки

Проверка mailcow выполняется через supported UI,
logs и docker compose ps

Reboot Test выявляет временные настройки

После перезагрузки должны сохраниться mount по UUID, IP forwarding, firewall и routes

OpenVPN, HAProxy, Zabbix Agent, Docker и Mailcow containers должны стартовать автоматически

Команда `systemctl is-enabled` проверяет автозапуск службы: `$ systemctl is-enabled haproxy`

Команда `mount` проверяет фактическое подключение файловых систем: `$ mount`

Проект не принимается, если работает только до первой перезагрузки

Полная остановка стенда требует порядка запуска

После shutdown всех VM сначала запускают DNS и базовую маршрутизацию

Затем поднимают storage, backup, zabbix01 и серверы приложений

После backend-сервисов запускаются HAProxy, Docker/Mailcow и клиенты

Если строгий порядок обязателен, он должен быть описан в документации

Идеальная инфраструктура выдерживает задержки зависимостей без ручной настройки каждой службы



Диаграмма: диагностика от клиента до данных

company.test — пошаговый путь поиска проблемы: от жалобы пользователя до сервиса, зависимостей и данных



Главный принцип

- 1 Идём от client path, а не от случайной перезагрузки
- 2 Сначала подтверждаем симптом
- 3 Потом проверяем ресурсы, сеть, службу и данные
- 4 После исправления обязательно повторяем пользовательский тест

1. Симптом у клиента



- Подтвердить жалобу
- Что именно недоступно?
- Когда началось?

2. DNS



- Имя резолвится?
- Проверка: `resolvectl query`
- `dig / nslookup`

3. Сеть / маршрут / firewall / VPN



- IP и маршрут
- Есть ли VPN?
- Открыт ли нужный порт?
- `nftables / ss`

4. Точка входа



- Доступен ли lb01?
- Отвечает ли HTTPS?
- Есть ли балансировка?

5. Служба / приложение



- `systemctl status`
- `journalctl`
- `nginx -t / haproxy -c`
- `docker compose ps`

6. Зависимости



- Какие сервисы нужны?
- Нет ли скрытой причины?
- Проверить цепочку зависимостей

7. Данные / хранилище



- Смонтирован ли том?
- Есть ли место?
- `df -h / df -i`
- Backup и restore

Базовые проверки ОС



`uptime` — load average



`free -h` — RAM / swap



`df -h` — место на ФС



`df -i` — inode



`systemctl --failed` — отказавшие службы

Команды по этапам



DNS

`resolvectl query lb01.company.test`



Network

`ip -br addr`

`ip route`

`ss -ltnup`

`nft list ruleset`



Service

`systemctl status nginx`

`journalctl -u nginx -n 50`

`nginx -t`

`haproxy -c -f /etc/haproxy/haproxy.cfg`



External test

`curl -I https://web.company.test`



Ожидаемый результат

- ✓ DNS-имя резолвится
- ✓ Маршрут и порт доступны
- ✓ curl даёт 200 / 301 или ожидаемый статус
- ✓ Сервис работает, данные доступны



Алгоритм исправления



Симптом



Проверка



Локализация



Исправление



Повторный тест



В отчёте фиксируются: симптом, гипотеза, команда, результат, исправление, подтверждение

Эксплуатационная инструкция описывает ежедневное сопровожден

Эксплуатационная инструкция нужна администратору, который будет обслуживать стенд после автора

Она описывает назначение сервиса, сервер, IP, зависимости, порты и конфигурационные файлы

В ней указываются ежедневные проверки, журналы, типовые предупреждения Zabbix и rollback

Команды должны иметь назначение, ожидаемый результат и способ интерпретации

Документ должен совпадать с фактически настроенной системой

шаблон эксплуатационной инструкции фиксирует одинаковый стандар

Раздел 1: назначение сервиса, владелец, сервер, IP и доменное имя

Раздел 2: зависимости, порты, firewall rules и конфигурационные файлы

Раздел 3: команды ежедневной проверки и нормальные признаки работы

Раздел 4: журналы, типовые alerts, безопасное изменение и rollback

Раздел 5: эскалация, ограничения и связанные runbooks

Recovery Runbook нужен для каждого критичного сервиса

Runbook — пошаговая инструкция восстановления после известного отказа

Минимум нужны runbooks для DNS, VPN, Web/HAProxy, Backup, Zabbix и Mailcow при полном варианте

Каждый runbook содержит Preconditions, Failure Action, Symptoms, Diagnosis, Fix, Recovery и Confirmation

Для restore указываются backup source, repository, команда восстановления и проверка сервиса

Одна демонстрация restore полезна, но для итоговой документации одного сервиса недостаточно

Исполнительная документация показывает фактический стенд

As-built documentation — описание того, что реально развернуто, а не что планировалось

Она включает логическую схему, схему интерфейсов, адресный план и DNS Zone Table

Нужны firewall matrix, список портов, certificates, volumes, mounts и backup schedule

Отдельно фиксируются версии ПО, Zabbix items/triggers и известные ограничения

Без as-built документации восстановление и проверка проекта становятся догадкой

Таблица конфигурационных файлов ускоряет проверку

OpenVPN: /etc/openvpn/server/server.conf и проверка подключения клиента

HAProxy: /etc/haproxy/haproxy.cfg и проверка \$ sudo haproxy -c -f /etc/haproxy/haproxy.cfg

Nginx: /etc/nginx/sites-enabled/ и проверка \$ sudo nginx -t

Docker: compose.yaml и проверка \$ docker compose config

Bash health check: /usr/local/sbin/server-health.sh и проверка \$ bash -n server-health.sh

Инвентаризация версий делает проект воспроизводимым

Фиксируются OS version, kernel, OpenVPN, HAProxy, Docker Engine и Docker Compose

Для полного варианта фиксируются Mailcow release, Zabbix Server/Agent и backup tool

Команда `uname -r` показывает версию ядра: `$ uname -r`

Команда `lsb_release -a` или `cat /etc/os-release` показывает версию дистрибутива

Несовпадение версий может сломать restore, update или повторную сборку стенда

Change Record связывает Git, backup и проверку

Change Record содержит дату, сервис, причину, исполнителя и измененные файлы

Перед изменением указываются backup, syntax check и ожидаемый rollback

После изменения фиксируются Git commit, команды проверки и результат monitoring

Для командного проекта запись показывает вклад участника и ответственность

Без Change Record трудно объяснить, почему система сейчас настроена именно так

Матрица ответственности нужна даже в учебном проекте

Для каждого сервиса указывается владелец настройки и владелец восстановления

Отдельно определяется, кто управляет CA, firewall, backup repository и Git

Root-доступ и доступ к backup не должны быть общими без учета

Командный проект оценивает не только результат, но и вклад участников

Ответственность помогает понять, кто объясняет сервис на защите

Реальные секреты запрещены во всех сдаваемых артефактах

Секреты не должны попадать в screenshots, logs, shell history, PCAP и отчеты

.env, private keys, OpenVPN profiles и backup passwords заменяются шаблонами

Вместо значения используется REDACTED, CHANGE_ME или SECRET_STORED_SEPARATELY

Преподаватель проверяет факт настройки без публикации секретного значения

Секреты в Git или отчете являются критической ошибкой безопасности

Пакет сдачи должен быть полным и согласованным

Схема сети, address plan, DNS records и firewall matrix должны ссылаться на одни имена

Git repository должен содержать scripts, sanitized configs, README и Change Record

Backup documentation должна показывать RPO, schedule, retention и test restore

Zabbix documentation должна показывать hosts, items, triggers и проверенные scenarios

Журнал неисправностей фиксирует symptoms, hypothesis, commands, fix и confirmation

Система оценивания переводит проект в 100 баллов

Архитектура, naming и address plan — 10 баллов

DNS, VPN, Web/HAProxy, Bash/Git и Storage — по 10 баллов

Backup and Restore — 15 баллов, потому что restore подтверждает ценность backup

Zabbix — 10 баллов; Docker/Mailcow или расширенная часть — 10 баллов

Документация и защита — 5 баллов при отсутствии критических нарушений

Критические условия неприятия проекта

Нет проверенного restore или студент не может объяснить процедуру восстановления

Реальные секреты опубликованы в Git, отчете, скриншотах или истории команд

Open Relay, полный admin-доступ из user network или отсутствующий firewall control

Невозможно объяснить address plan, DNS role или зависимости сервисов

Сервисы работают случайно и не восстанавливаются после reboot

Приемочные тесты штатной работы

DNS lookup всех серверов возвращает адреса из address plan

VPN valid certificate устанавливает tunnel, а внутренний web открывается через lb01

HAProxy распределяет трафик между web01 и web02
server-health.sh возвращает понятный output и корректный exit code

Mailcow в полном варианте доставляет письмо user1@company.test -> user2@company.test

Приемочные тесты безопасности

VPN user может открыть разрешенный web, но не может зайти по SSH на backend

Revoked VPN certificate не устанавливает tunnel

Management UI доступны только с admin01 или management network

В Git и отчете нет реальных секретов, private keys и паролей

Firewall matrix совпадает с фактическими nftables или другими правилами

Приемочные тесты отказа и восстановления

Остановка web01 переводит backend в DOWN, но web продолжает работать через web02

Zabbix показывает problem и закрывает его после recovery

Test restore восстанавливает выбранный сервис и фиксирует фактический RTO

Docker container пересоздается без потери persistent data

Reboot всех ключевых узлов сохраняет работоспособность сервисов



Инфографика: матрица приемки итогового Linux-проекта

company.test — что именно проверяется на защите, по каким критериям и что ведёт к непринятию проекта



1. Основные блоки оценки

Блок	Что проверяют	Баллы
Архитектура и naming	Роли, адресный план, DNS, зависимости	10
Базовые сервисы	DNS, VPN, Web/HAProxy, Bash/Git, Storage	по 10
Backup и Restore	Backup, test restore, RPO/RTO	15
Monitoring / Zabbix	Hosts, items, triggers, свежесть данных	10
Документация и защита	Схемы, matrix, runbooks, объяснение решений	5

Расширенная часть (Docker / Mailcow и др.) может оцениваться отдельно.

2. Приемочные тесты штатной работы

- DNS lookup всех узлов возвращает адреса из address plan
- VPN с valid certificate поднимает tunnel
- Web открывается через lb01
- HAProxy распределяет трафик между web01 и web02
- server-health.sh даёт понятный output
- Mailcow доставляет письмо user1 → user2 (если входит в вариант проекта)

3. Приемочные тесты безопасности

- VPN user видит только разрешённые ресурсы
- SSH на backend запрещён из user/VPN network
- Revoked certificate не подключается
- Management UI доступны только из admin01 / MGMT
- Firewall matrix совпадает с реальными правилами
- В Git и отчёте нет реальных секретов

4. Тесты отказа и восстановления



5. Критические условия непринятия

- Нет проверенного restore или студент не может его объяснить
- Секреты опубликованы в Git, отчёте или скриншотах
- Open Relay / лишний admin-доступ / отсутствующий firewall control
- Невозможно объяснить address plan, DNS role или зависимости
- Сервисы работают случайно и не переживают reboot

Что сдаёт студент

- Схема сети
- Address plan
- Firewall matrix
- Runbooks / Change Record

Финальный вывод



Проект принимается только тогда, когда он работает как связанная система, проходит тесты и может быть восстановлен по документации.

Итоги лекции

Итоговый проект оценивает связанную инфраструктуру, а не отдельные установленные службы

DNS, addressing, firewall matrix и dependency matrix делают стенд проверяемым

Git, backup и monitoring должны появиться до сложных изменений, а не после проблем

Диагностика идет от client path и базовых ресурсов к службам, приложениям и данным

Проект считается завершенным только после acceptance tests, reboot test и проверенного restore

Контрольные вопросы

Почему в итоговом проекте нужно явно определить, где находится DNS-сервер?

Чем отличается production traffic от management traffic и backup traffic?

Почему отсутствие Zabbix problems не всегда доказывает исправность сервиса?

Какие проверки нужно выполнить перед Git commit конфигурационного изменения?

Почему backup без test restore нельзя считать достаточным результатом?

Как проверить, что VPN дает разрешенный доступ и